Dr. P. Raghu Vamsi

* THE Greedy method:-

Greedy method is a straight forward design technique that can be applied to a varity of problems. In this problem have "n" i/p's and require us to obtain a subset that satisfies these constraints is called a feasible solution. we need to feasible solution that either maximize (or) minimize a given objective. [function] f^n.

A feasible solution that does this is called optimal solution.

Eg:-
Knapsack problem.

This is the drawback of divide-and-conquer technique, which is rectified in greedy method.

Greedy Algorithm:-

```
Algorithm Greedy(a,n)
// a[1:n] contains the n i/ps
{
  Solution = ∅;   // initialize solution
  for i=1 to n do
  {
    x = Select (a);
    if feasible (Solution, x) then
```

```
    solution = union (solution, x);
  }
  return solution;
}
```

The important steps for achieving Greedy algorithms are.

1) feasible:- Here we check whether it satisfies all the possible problem constraints or not, so as to obtain atleast one solution to our problem.

2) Local optimal choice:-

In this, choice should be optimum which is selected from the current available feasible choices.

3) unalterable:-

once that choice is made, at any subsequent step that choice is not altered.

Applications:-

Job sequencing with Dead lines:-

statement:-

we have on a single process computer "n" Jobs are submitted ($j_1, j_2, j_3 \ldots j_n$) with known running times ($T_1, T_2, \ldots T_n$), here the problem is to schedule

these Jobs in a order to minimize the average waiting time.

To formulate a greedy algorithm to obtain an optimal solution, it is necessary to formulate an optimization measure to determine how the next job is choosen. The objective function $\sum_{i \in J} p_i$ is choosen as an optimization measure.

using this measure, the next job to be included is the one that increases $\sum_{i \in J} p_i$ the most, subject to the constraint that the resulting 'J' is a feasible solution. this requires the Jobs to be considered in non-increasing order of the p_i's. initially $J=0$ and $\sum_{i \in J} p_i = 0$

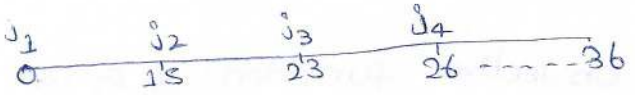
Next, the Job that has the largest profit is added. it forms feasible soln. The next Job, to Join the set(J), has to satisfy the following conditions

1) The Job should have maximum profit so as to form an optimal soln.

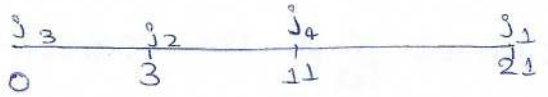
2) The Job to be included should be completed by it's deadline to form a feasible soln.

Ex:-

4 Jobs are submitted j_1, j_2, j_3, j_4 & their execution times are 15, 8, 3, 10 respectively if we allow the Jobs based on their arrival then the average waiting time will be



$$\frac{0+15+23+26}{4} = 16$$ (but its not an optimal soln)



$$\frac{0+3+11+21}{4} = \frac{35}{4} = 8.75.$$

The optimization is acheived by executing a Jobs in the ascending order of their required time in this case average waiting is minimum

No other arrangement will give the soln less that 8.75 so the optimal sequence is j_3, j_2, j_4, j_1.

Dr. P. Raghu Vamsi

* Greedy knapsack problem:-

we were given "n" objects & a knapsack (or) a bag. object "i" has a weight w_i, & knapsack has a capacity "m". if a fraction x_i, $0 \le x_i \le 1$ of object "i" is placed into the knapsack. then a profit p_i & x_i is earned. the object is to obtain a filling of the knapsack that maximizes the total profit earned.

$$\text{maximize} \sum_{1 \le i \le n} p_i x_i$$

$$\text{Subject to} \sum_{1 \le i \le n} w_i x_i \le m \quad \text{and} \quad 0 \le x_i \le 1,\ 1 \le i \le n$$

Eg:-

consider the following instance of the knapsack problem $n = 3$ $m = 20$

$$(P_1, P_2, P_3) = (25, 24, 15)$$

$$(w_1, w_2, w_3) = (18, 15, 10)$$

Soln: Case-1:-

$$(x_1, x_2, x_3) = (1/2, 1/3, 1/4)$$

$$\sum w_i x_i = \overset{9}{18} \times 1/2 + \overset{5}{15} \times 1/3 + \overset{2.5}{10} \times 1/4$$

$$= 16.5$$

$$\sum p_i x_i = 25 \times 1/2 + 24 \times 1/3 + \overset{2.7}{\cancel{15}} \times 1/4$$

$$= 24.25$$

case - ii :-

$$(x_1, x_2, x_3) = (1, 2/15, 0)$$

$$\sum w_i x_i = \left(18 \times 1 + \cancel{15} \times \frac{2}{\cancel{15}} + 10 \times 0\right)$$

$$= 18 + 2 = 20$$

$$\sum p_i x_i = 25 \times 1 + 24 \times 2/15 + 15 \times 0$$

$$= 28.2$$

case - iii :-

$$(x_1, x_2, x_3) = (0, 2/3, 1)$$

$$\sum w_i x_i = 18 \times 0 + 1\overset{5}{\cancel{8}} \times 2/\cancel{3} + 10 \times 1$$

$$= 10 + 10 = 20$$

$$\sum p_i x_i = 25 \times 0 + 24 \times 2/3 + 10 \times 1$$

$$= 31$$

case iv :-

$$(x_1, x_2, x_3) = (1, 1/2)$$

$$\sum w_i x_i = 18 \times 0 + 15 \times 1 + 1\overset{5}{\cancel{0}} \times 1/2$$

$$= 15 + 5 = 20$$

$$\sum p_i x_i = 25 \times 0 + 24 \times 1 + 1\overset{5}{\cancel{0}} \times 1/2 = 31.5$$

→ In 1st case we have selected in different fractions & it lead to the weight 16.5 & profit 24.25

→ In the above four cases fourth case is suitable because it has wt = 20 & profit also high 31.5 remaining to all.

→ The optimal solution is in 4th case.

* Algorithm:-

```
Algorithm GreedyKnapsack(m,n)
{
  for i=1 to n do x[i]=0.0;
  U=m;
  for i=1 to n do {
      if (w[i] >U) then break;
      x[i] =1.0 ; U = U-w[i];
  }
  if (i≤n) then x[i]:=U/w[i];
}
```

→ The time complexity of Greedy Knapsack problem is "O(n)".

* Construction of minimal spanning Trees:-

Let "G" be a connected graph. A subgraph "T" of "G" is called a spanning tree of G if it satisfies the following conditions.

1) T is a tree
2) T contains all vertices of G.

The edges are called "branches". If a graph "G" has "n" vertices, then a spanning tree of "G" should have n vertices & n-1 edges.

In this we have two algorithms.

1) prims algorithm
2) krushkal's algorithm.

A minimal spanning tree is a tree constructed by using the edges from the graph such that the total weight of all the edges is least. using the vertices & edges of the given graph we construct a tree which is connected. such that the total weight of these edges is minimum.

The newly constructed tree is called "minimal" (or) "optimal spanning tree".

→ Krushkal's algorithm for optimal spanning tree:-

Step-1:- arrange all the edges in the ascending order of their wts & call this set as "D"

Step-2:-

Select the edge from the set "D" when as the least weight, remove it from D & place it in the tree. If this placing don't form a cycle continue. else neglect the edge to place in the tree.

Step-3:-

Take the next edge with least weight from the edge set "D" & continue this procedure using Step-2.

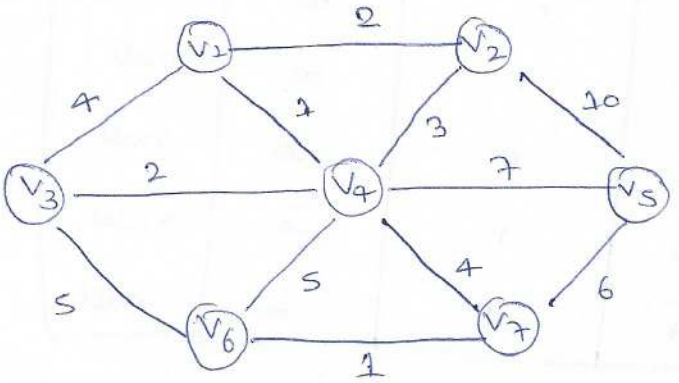
Step-4:-

Continue this procedure until the set "D" is empty.

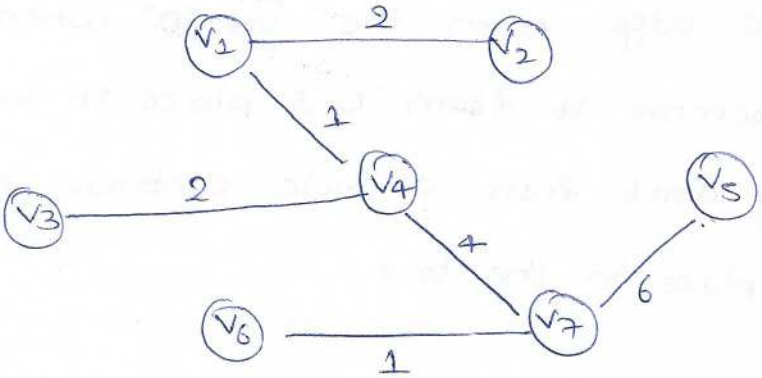
Step-5:-

for "n" vertices of this graph if there are n-1 edges then the constructed spanning tree is correct.

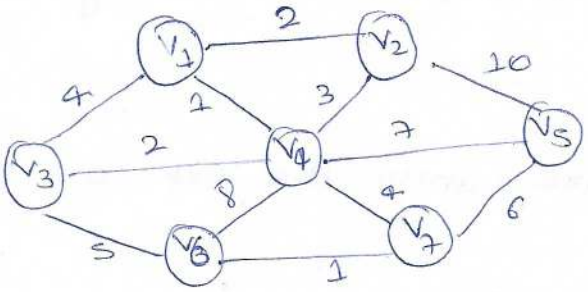
Eg:-



$(V_1, V_4) = 1$	$(V_6, V_7) = 1$	$(V_1, V_2) = 2$	$(V_3, V_4) = 2$
$(V_2, V_4) = 3$	$(V_4, V_7) = 4$	$(V_1, V_3) = 4$	$(V_3, V_6) = 5$
$(V_5, V_7) = 6$	$(V_4, V_5) = 7$	$(V_4, V_6) = 8$	$(V_2, V_5) = 10$



* Prim's Algorithm for optimal Spanning tree:-



vertex	visited	weights	path.
V_1	F	∞	Null
V_2	F	∞	null
V_3	F	∞	null
V_4	F	∞	null
V_5	F	∞	null
V_6	F	∞	null
V_7	F	∞	null

→ create a table using the columns vertex, visited, weights & path. the initial value in the path are Null, weights are "∞", and visited are false.

→ select any vertex whose edge with least weight select V_1 & assign visited are true. update all adjacent vertices with least weight

vertex	visited	weights	path
V_1	T	∞	null
V_2	F	2	V_1
V_3	F	4	V_1
V_4	F	1	V_1
V_5	F	∞	null
V_6	F	∞	null
V_7	F	∞	null

vertex	visited	weights	path
V_1	T	∞	null
V_2	F	2	V_1
V_3	F	2	V_4
V_4	T	1	V_4
V_5	F	7	V_4
V_6	F	8	V_4
V_7	F	4	V_4

→ select the vertex v_4 and update the adjacent vertices with least weight in the above table.

→ select the vertex v_2 and update the adjacent vertices with least weight.

vertex	visited	weights	path
v_1	T	∞	null
v_2	T	2	v_1
v_3	F	2	v_4
v_4	T	1	v_1
v_5	F	7	v_4
v_6	F	8	v_4
v_7	F	4	v_4

→ select the vertex v_3 & update the adjacent vertices with least weight.

vertex	visited	weights	path
v_1	T	4	null
v_2	T	2	v_1
v_3	T	2	v_4
v_4	T	1	v_1
v_5	F	7	v_4
v_6	F	5	v_3
v_7	F	4	v_4

→ Select the vertex v_7 and update.

vertex	visited	weights	path
v_1	T	∞	null
v_2	T	2	v_1
v_3	T	2	v_4
v_4	T	1	v_1
v_5	F	6	v_7
v_6	F	1	v_7
v_7	T	4	v_4

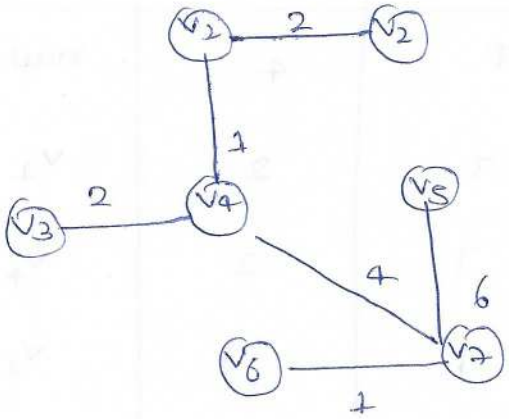
→ Select v_6 & update.

vertex	visited	weights	path
v_1	T	4	null
v_2	T	2	v_1
v_3	T	2	v_4
v_4	T	1	v_1
v_5	F	6	v_7
v_6	T	1	v_7
v_7	T	1	v_4

Select v_5 & update

vertex	visited	weights	path
v_1	T	∞	null
v_2	T	2	v_1
v_3	T	2	v_4
v_4	T	1	v_1
v_5	T	6	v_7
v_6	T	1	v_7
v_7	T	4	v_4

$\{(v_2,v_1),(v_3,v_4),(v_4,v_1),(v_5,v_7),(v_6,v_7),(v_7,v_4)\}$



Note:- minimal spanning tree is same for both krushkals & prims algorithm.

→ The time complexity of krushkal's & prim's algorithm is "$O(n^2)$".

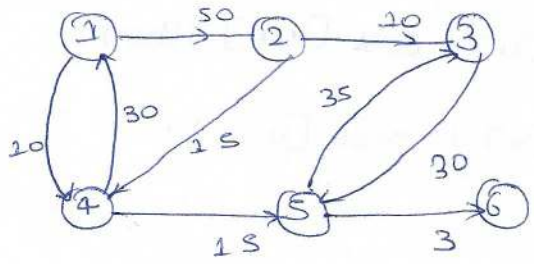
```
Algorithm prims (E, cost, n, t) {
Let (k,l) be an edge of minimum cost in E;
min cost := cost (k,l);
t[1,1] = k ; t[1,2] = l;
for i = 1 to n do
  if (cost [i,l] < cost [i,k]) then near [i] = l;
  else near[i] = k;
near [k] = near[l] = 0;
for i = 2 to n-1 do {
Let j be an index such that near [j] ≠ 0 and
cost [j, near[j]] is minimum;
t[i,1] = j; t[i,2] = near [j];
mincost = mincost + cost [ ], near [j]];
near [j] = 0
for k = 1 to n do
{
  if ((near[k] ≠ 0) & (cost[k, near[k]] > cost [k,j]));
   then near[k] = j;
}
return mincost;
}
```

```
Algorithm krushkal (E, cost, n, t){
  for i=1 to n do parent[i]=-1;
  i=0; mincost=0;
  while ((i<n-1) and (heapnot empty)) do
  {
    Delete a minimum cost edge (u,v) from
    the heap and reheapify using adjust;
    j=find(u); k=find(v);
    if (j≠k) then {
      i=i+1;
      t[i,1]=u; t[i,2]=v;
      mincost = mincost + cost[u,v];
      union(j,k);
    }
  }
  if (i≠n-1) then write("No spanning tree");
  else return mincost;
  }
}.
```

* Single source shortest path:-

It is applied only for directed graph

eg:-



Source	destination
1, 4	10
1, 4, 5	25
1, 4, 5, 2	45
1, 3	45

Shortest path from 1.

Algorithm:-

```
Algorithm shortestpath(v, cost, dist, n){
for i=1 to n {
    S[i] = false; dist[i] = cost[v,i];
}
S[v] = true ; dist[v] = cost[v,i];
for num=2 to n do {
    choose u from among those vertices not in S
    such that dist[u] is minimum;
```

```
S[u] = true;
for (each w adjacent to u with S[w] = false )
do {
    if (dist[w] > dist[u] + cost [u,w] ) then
        dist [w] = dist [v] + cost [u,w];
    }
}
```

→ Time complexity is "$O(n^2)$";